Andrés Merino • Abril 2026

Librerías necesarios

Importamos las librerías necesarias:

- **numpy**: Para operaciones numéricas
- **IPython.display**: Para mostrar contenido formateado (Markdown, LaTeX)
- **qiskit**: Librería de computación cuántica
- **plotly**: Para crear gráficas interactivas 3D
- **ipywidgets**: Para crear interfaces interactivas en el notebook

In [1]:
```
import numpy as np
from IPython.display import Markdown, display
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
import plotly.graph_objects as go
import ipywidgets as widgets
```

Configuramos el renderer de Plotly para que las gráficas se muestren correctamente en el notebook exportado.

In [2]:
```
import plotly.io as pio
pio.renderers.default = 'notebook'
```

Pautas de elaboración

Diseñar e implementar una simulación interactiva que permita explorar el espacio de estados de un cúbit. La actividad debe incluir la visualización del cúbit en la esfera de Bloch y la demostración de cómo las puertas cuánticas afectan su estado.

La esfera debe ser capaz de representar cualquier estado cuántico. Una vez representado dicho estado, la simulación debe ser capaz de mostrar el resultado que provocarían las siguientes puertas:

- Puerta Hadamard.
- Puertas Paulis (X, Y, Z, I).
- Puertas S y T, así como sus adjuntas.
- Puerta V (también llamada raíz cuadrada de la puerta X) y su adjunta.

La simulación debe ser compatible con la librería Qiskit, de forma que el estado cuántico a representar se le pase a través de dicha librería. La esfera debe ser programada por el estudiante. No es válido el uso de esferas ya incluidas en otras librerías (incluyendo la propia librería Qiskit).

Se debe aportar diferentes ejemplos resueltos que sean suficientes para demostrar el buen funcionamiento del código, así como el propio cuaderno impreso en PDF o en HTML. Es responsabilidad del estudiante que el cuaderno impreso se pueda visualizar correctamente.

Implementación

Definición de constantes:

- **GATE_METHODS**: Diccionario de las puertas cuánticas
  - I: Identidad
  - X, Y, Z: Puertas de Pauli
  - H: Hadamard
  - S, Sdg: Puerta S y su adjunta
  - T, Tdg: Puerta T y su adjunta
  - V, Vdg: Puerta V (√X) y su adjunta
- **COLOR**: Color los elementos gráficos

In [3]:
```
GATE_METHODS = {
    "I": "id",
    "X": "x",
    "Y": "y",
    "Z": "z",
    "H": "h",
    "S": "s",
    "Sdg": "sdg",
    "T": "t",
    "Tdg": "tdg",
```

```
    "V": "sx",
    "Vdg": "sxdg",
}

COLOR = "rgba(100,200,255,0.35)"
```

Funciones auxiliares:

1. **ket_to_bloch(ket)**: Convierte un estado cuántico (ket) a coordenadas en la esfera de Bloch
   - Entrada: Vector de estado cuántico
   - Salida: Coordenadas (x, y, z) en la esfera de Bloch
   - Usa la fórmula:

$$x = 2\Re(\alpha^* \beta), \quad y = 2\Im(\alpha^* \beta), \quad z = |\alpha|^2 - |\beta|^2$$

2. **angles_to_ket(theta, phi)**: Crea un estado cuántico a partir de dos ángulos θ y ϕ
   - Parametrización estándar de la esfera de Bloch
   - El estado resultante es:

$$\cos(\theta/2)|0\rangle + e^{i\phi} \sin(\theta/2)|1\rangle$$

3. **plot_bloch_sphere(ket_in, ket_out, gate_name)**: Visualiza dos estados en la esfera de Bloch
   - Muestra el estado inicial (verde) y el estado final después de aplicar una puerta (rojo)
   - Dibuja la esfera, los ejes principales, y los estados como vectores

In [4]:
```
# Función que toma un ket Statevector y devueve las coordenadas de la esfera de Bloch
def ket_to_bloch(ket):
    ket = ket / np.linalg.norm(ket.data)
    a, b = ket.data
    x = 2 * np.real(np.conj(a) * b)
    y = 2 * np.imag(np.conj(a) * b)
    z = np.abs(a)**2 - np.abs(b)**2
    return float(x), float(y), float(z)

# Función de ángulos a ket Statevector
def angles_to_ket(theta, phi):
    alpha = np.cos(theta / 2)
    beta = np.exp(1j * phi) * np.sin(theta / 2)
    return Statevector([alpha, beta])

# Función que toma dos kets Statevector y una puerta, y grafica ambos kets en la esfera de Bloch
def plot_bloch_sphere(ket_in, ket_out, gate_name="U"):
    x1, y1, z1 = ket_to_bloch(ket_in)
    x2, y2, z2 = ket_to_bloch(ket_out)

    traces = []

    # Esfera
    u = np.linspace(0, 2 * np.pi, 60)
    v = np.linspace(0, np.pi, 60)
    sx = np.outer(np.cos(u), np.sin(v))
    sy = np.outer(np.sin(u), np.sin(v))
    sz = np.outer(np.ones_like(u), np.cos(v))

    traces.append(go.Surface(
        x=sx, y=sy, z=sz,
        opacity=0.08,
        colorscale=[[0, "#1a1a2e"], [1, "#16213e"]],
        showscale=False,
        hoverinfo="skip",
        name="Esfera"
    ))

    # Ecuador
    eq = np.linspace(0, 2 * np.pi, 120)
    traces.append(go.Scatter3d(
        x=np.cos(eq), y=np.sin(eq), z=np.zeros_like(eq),
        mode="lines",
        line=dict(color=COLOR, width=1.5),
        hoverinfo="skip",
        showlegend=False
    ))
    # Dos Meridianos
    mer1 = np.linspace(0, 2*np.pi, 60)
    traces.append(go.Scatter3d(
        x=np.cos(mer1), y=np.zeros_like(mer1), z=np.sin(mer1),
        mode="lines",
        line=dict(color=COLOR, width=1.5),
        hoverinfo="skip",
        showlegend=False
    ))
    mer2 = np.linspace(0, 2*np.pi, 60)
    traces.append(go.Scatter3d(
        x=np.zeros_like(mer2), y=np.cos(mer2), z=np.sin(mer2),
        mode="lines",
        line=dict(color=COLOR, width=1.5),
        hoverinfo="skip",
        showlegend=False
```

```
    ))

    # Ejes
    axis_len = 1.2
    axes = [
        (0, 0, axis_len, 0, 0, -axis_len, "|0⟩", "|1⟩"),
        (axis_len, 0, 0, -axis_len, 0, 0, "|+⟩", "|-⟩"),
        (0, axis_len, 0, 0, -axis_len, 0, "|+i⟩", "|-i⟩"),
    ]
    for x, y, z, xn, yn, zn, lab1, lab2 in axes:
        traces.append(go.Scatter3d(
            x=[xn, x], y=[yn, y], z=[zn, z],
            mode="lines+text",
            line=dict(color=COLOR, width=3),
            text=[lab2, lab1],
            textposition="top center",
            textfont=dict(size=14, color=COLOR),
            hoverinfo="skip",
            showlegend=False
        ))

    # Ket entrada
    traces.append(go.Scatter3d(
        x=[0, x1], y=[0, y1], z=[0, z1],
        mode="lines",
        line=dict(color="#2ed573", width=5),
        hoverinfo="skip",
        showlegend=False
    ))
    traces.append(go.Scatter3d(
        x=[x1], y=[y1], z=[z1],
        mode="markers",
        marker=dict(size=5, color="#2ed573", line=dict(color="white", width=1)),
        name="Entrada |ψ⟩",
    ))

    # Ket salida
    traces.append(go.Scatter3d(
        x=[0, x2], y=[0, y2], z=[0, z2],
        mode="lines",
        line=dict(color="#ff4757", width=5),
        hoverinfo="skip",
        showlegend=False
    ))
    traces.append(go.Scatter3d(
        x=[x2], y=[y2], z=[z2],
        mode="markers",
        marker=dict(size=5, color="#ff4757", line=dict(color="white", width=1)),
        name=f"Salida {gate_name}|ψ⟩",
    ))

    # Configuración de la figura
    fig = go.Figure(data=traces)
    fig.update_layout(
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)",
        scene=dict(
            bgcolor="rgba(8,8,20,1)",
            aspectmode="cube",
            xaxis=dict(showticklabels=False, showgrid=False, zeroline=False, showbackground=False, range=[-1.4, 1.4]),
            yaxis=dict(showticklabels=False, showgrid=False, zeroline=False, showbackground=False, range=[-1.4, 1.4]),
            zaxis=dict(showticklabels=False, showgrid=False, zeroline=False, showbackground=False, range=[-1.4, 1.4]),
            camera=dict(eye=dict(x=1.5, y=1.5, z=0.8)),
        ),
        margin=dict(l=0, r=0, t=30, b=0),
        height=500,
        legend=dict(
            x=0.01, y=0.98,
            font=dict(color="white", size=12),
            bgcolor="rgba(0,0,0,0.4)",
            bordercolor="rgba(255,255,255,0.2)",
            borderwidth=1,
        ),
    )

    fig.show()
```

Ejemplo no interactivo

Ejemplo 1: Entrada $|0\rangle$ y aplicación de la puerta X

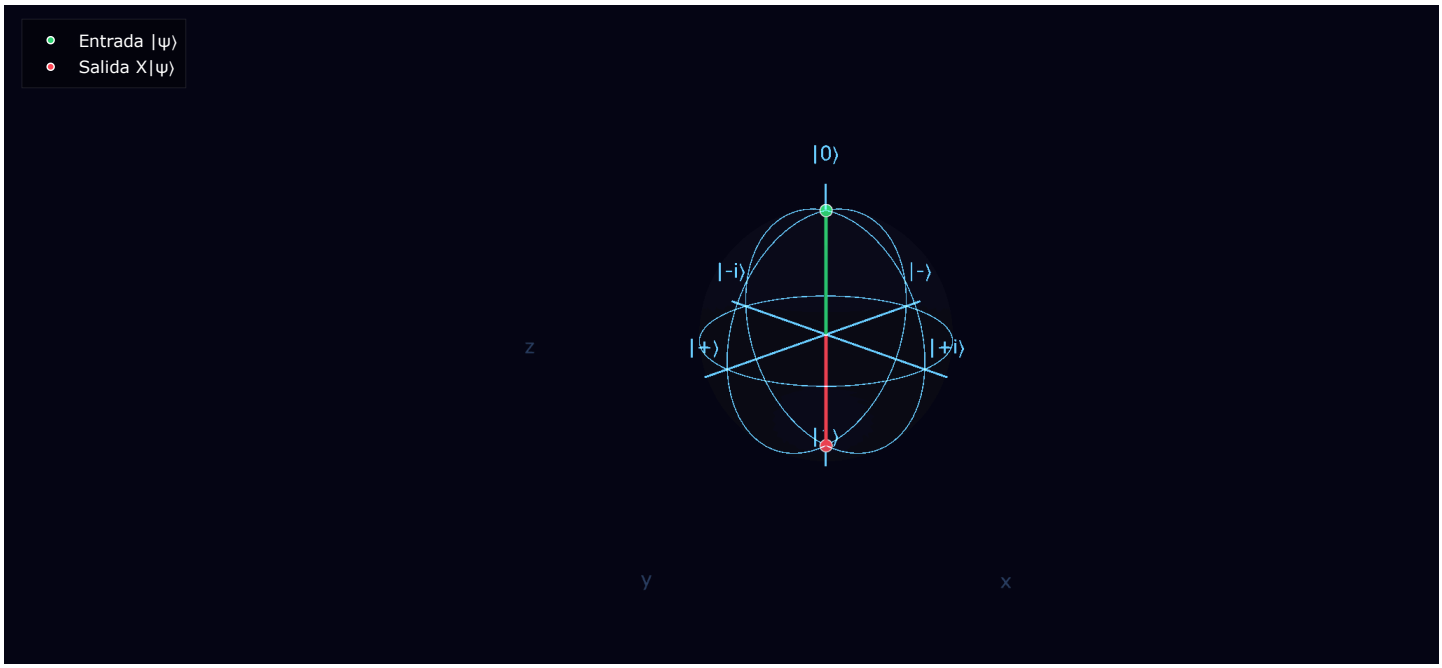
1. Definimos ángulos $\theta = 0$ y $\phi = 0$ (estado $|0\rangle$)
2. Mostramos la representación del estado inicial
3. Dibujamos el circuito cuántico con la puerta X
4. Calculamos el estado resultante: $X|0\rangle = |1\rangle$
5. Visualizamos ambos estados en la esfera de Bloch

In [5]:
```
# Ángulos
theta = 0
```

```
phi = 0

# Ket ingresado por el usuario
ket = angles_to_ket(theta, phi)
display(ket.draw(output="latex"))

# Puerta a aplicar
qc = QuantumCircuit(1)
qc.x(0)
display(qc.draw(output="mpl"))

# Ket resultante
ket_res = ket.evolve(qc)
display(ket_res.draw(output="latex"))

# Visualización en la esfera de Bloch
plot_bloch_sphere(ket, ket_res, "X")
```

$|0\rangle$

$|1\rangle$



Ejemplo 2: Entrada $\frac{\sqrt{2}}{2}(|0\rangle + i|1\rangle)$ y aplicación de la puerta $T^\dagger$

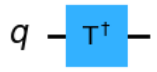
En este ejemplo:

1. Usamos ángulos $\theta = \pi/2$ y $\phi = \pi/2$ (estado $\frac{\sqrt{2}}{2}(|0\rangle + i|1\rangle)$)
2. Mostramos la representación del estado inicial
3. Dibujamos el circuito cuántico con la puerta $T^\dagger$
4. Calculamos el estado resultante: $T^\dagger\left(\frac{\sqrt{2}}{2}(|0\rangle + i|1\rangle)\right)$
5. Visualizamos ambos estados en la esfera de Bloch

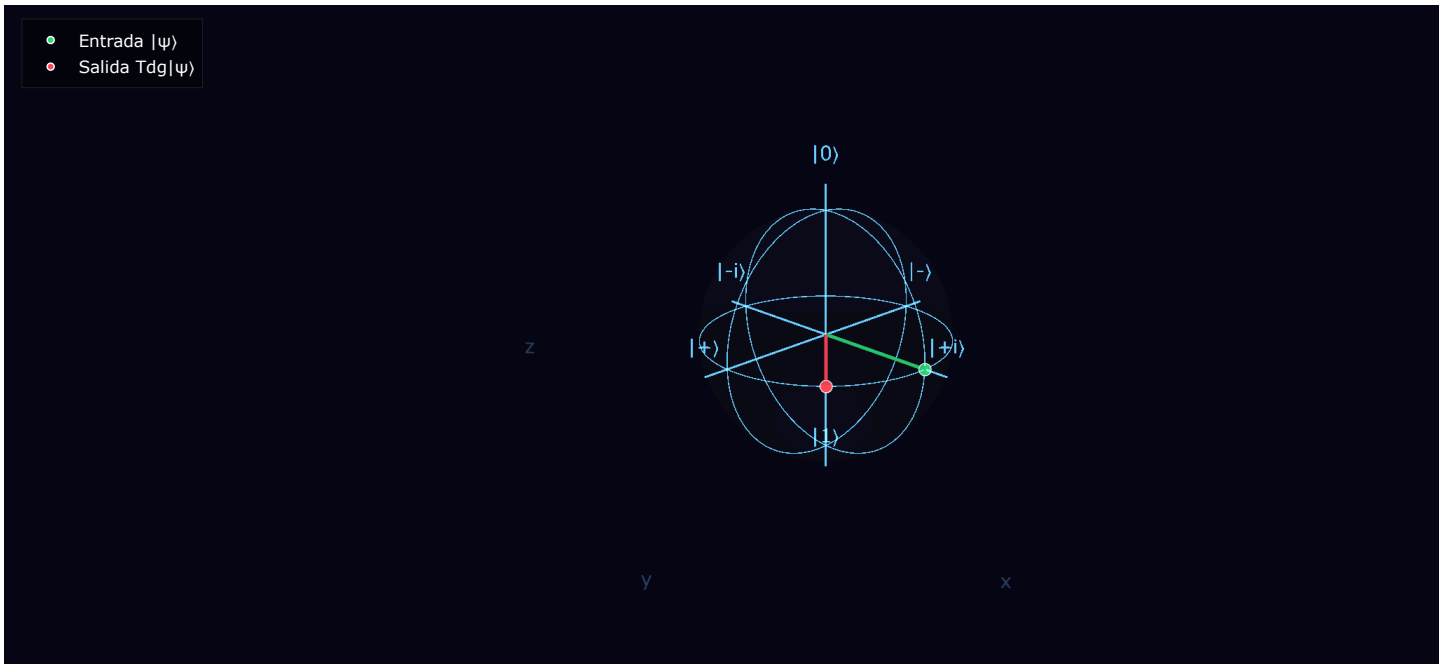
In [6]:
```
# Ángulos
theta = np.pi/2
phi = np.pi/2

# Ket ingresado por el usuario
ket = angles_to_ket(theta, phi)
display(ket.draw(output="latex"))

# Puerta a aplicar
qc = QuantumCircuit(1)
qc.tdg(0)
display(qc.draw(output="mpl"))

# Ket resultante
ket_res = ket.evolve(qc)
display(ket_res.draw(output="latex"))
```

```
# Visualización en la esfera de Bloch
plot_bloch_sphere(ket, ket_res, "Tdg")
```

$$\frac{\sqrt{2}}{2}|0\rangle + \frac{\sqrt{2}i}{2}|1\rangle$$



$$\frac{\sqrt{2}}{2}|0\rangle + (\frac{1}{2} + \frac{i}{2})|1\rangle$$



Interactividad

En las siguientes celdas se define una interfaz interactiva que permite:

- Ajustar los ángulos θ y ϕ usando deslizadores
- Seleccionar cualquier puerta cuántica del menú desplegable
- Ver la representación del estado inicial y final en la esfera de Bloch

Esto facilita la exploración y comprensión de cómo las puertas cuánticas transforman los estados en un cúbit.

La función `mostrar()` realiza los siguientes pasos:

1. Crea un estado cuántico a partir de los ángulos θ y ϕ
2. Construye un circuito cuántico con la puerta seleccionada
3. Evoluciona el estado aplicando la puerta
4. Muestra el estado inicial
5. Dibuja el circuito cuántico
6. Muestra el estado resultante
7. Visualiza ambos estados en la esfera de Bloch

Los widgets permiten cambiar todos estos parámetros y ver los resultados.

In [7]:
```
# Función que toma dos kets Statevector y una puerta, y grafica ambos kets en la esfera de Bloch
def mostrar(theta, phi, puerta):
    # Ket ingresado por el usuario
    ket = angles_to_ket(theta, phi)
    # Puerta a aplicar
    qc = QuantumCircuit(1)
    getattr(qc, GATE_METHODS[puerta])(0)
    # Ket resultante
    ket_res = ket.evolve(qc)

    # Visualización
    display(Markdown(f'**Ket inicial** para `theta={theta:.2f}` y `phi={phi:.2f}`'))
    display(ket.draw(output='latex'))

    display(Markdown(f'**Puerta seleccionada:** `{puerta}`'))
```

```python
    display(qc.draw(output='mpl'))

    display(Markdown('**Ket resultante**'))
    display(ket_res.draw(output='latex'))

    plot_bloch_sphere(ket, ket_res, puerta)

# Widgets para ángulos y selección de puerta
theta_widget = widgets.FloatSlider(
    value=0,
    min=0,
    max=np.pi,
    step=0.01,
    description='theta',
    continuous_update=False,
    readout_format='.2f'
)

phi_widget = widgets.FloatSlider(
    value=0,
    min=0,
    max=2 * np.pi,
    step=0.01,
    description='phi',
    continuous_update=False,
    readout_format='.2f'
)

gate_widget = widgets.Dropdown(
    options=list(GATE_METHODS.keys()),
    value='H',
    description='Puerta'
)

# Salida interactiva
salida_interactiva = widgets.interactive_output(
    mostrar,
    {'theta': theta_widget, 'phi': phi_widget, 'puerta': gate_widget}
)

# Mostrar widgets y salida
display(widgets.VBox([
    widgets.HTML('<b>Explora un cúbit sobre la esfera de Bloch</b>'),
    theta_widget,
    phi_widget,
    gate_widget
]))
display(salida_interactiva)
```

Explora un cúbit sobre la esfera de Bloch

theta 0.00

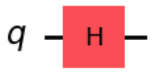
phi 0.00

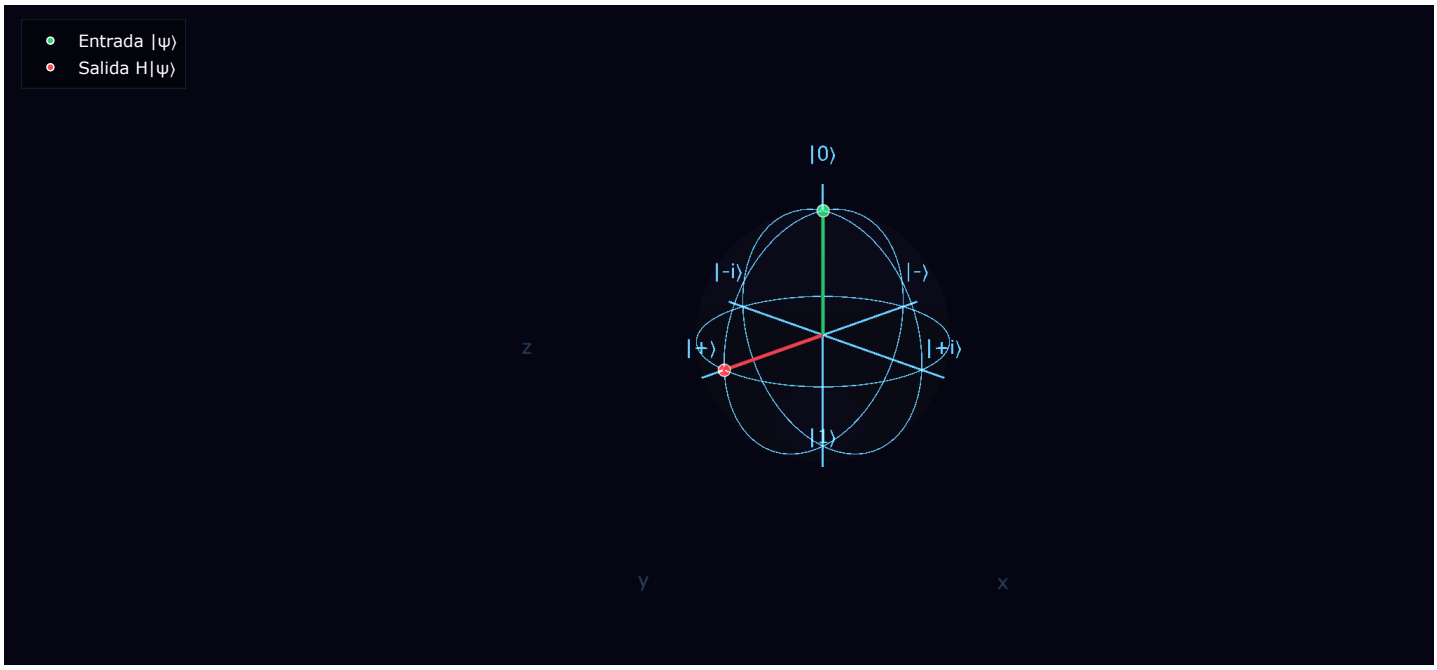
Puerta H

Ket inicial para `theta=0.00` y `phi=0.00`

$$ |0\rangle$$

Puerta seleccionada: `H`



Ket resultante

$$\frac{\sqrt{2}}{2} |0\rangle+\frac{\sqrt{2}}{2} |1\rangle$$